

WEB DEVELOPMENT FUNDAMENTALS

Basic Web Architecture

Some Slides drafted by Claude (July 2026)

Web Architecture คืออะไร?

การออกแบบ Web Architecture คือ การกำหนดว่า “เว็บแอปพลิเคชันจะถูกแบ่งส่วนและทำงานร่วมกันอย่างไร” เพื่อให้ระบบมีคุณภาพที่ดีในด้านต่าง ๆ

1

ความเร็ว

Speed

2

ความปลอดภัย

Security

3

การขยายระบบ

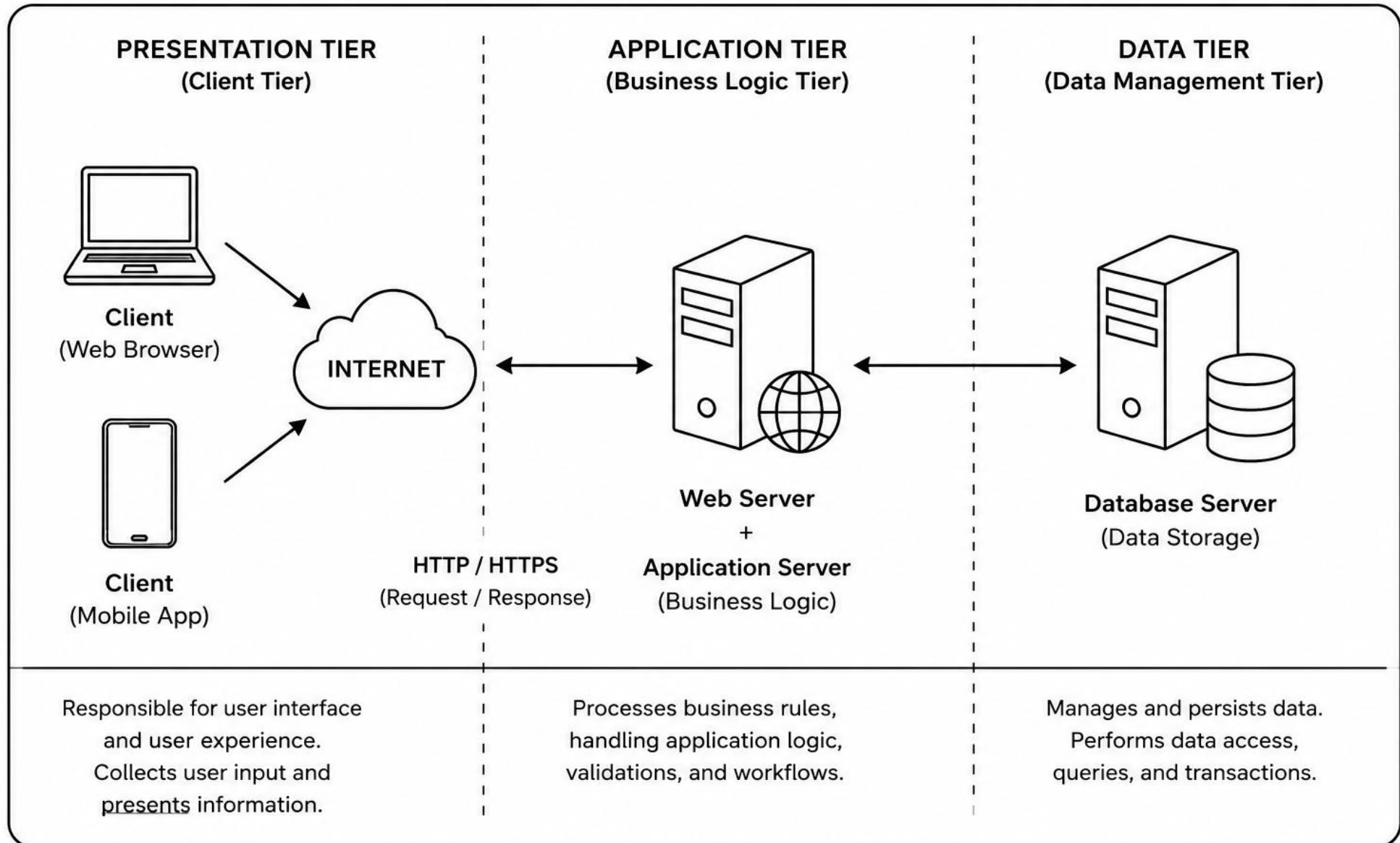
Scalability

4

การบำรุงรักษา

Maintainability

3 Tier Web Architecture (Presentation Tier – Application Tier – Data Tier)



หน้าที่ของ **Client-Server**

CLIENT (BROWSER)

- Loads HTML, CSS, and JavaScript on first request
- Renders the visual interface the user sees
- Listens for user events (clicks, input, scroll)
- Calls the Fetch API to talk to the server
- Updates the DOM when new data arrives

SERVER (BACKEND API)

- Receives HTTP requests from the client
- Applies business logic and authentication
- Reads or writes data in a database
- Formats a response as JSON
- Sends the response back to the browser

องค์ประกอบหลักของเว็บแอปพลิเคชัน



Client-Side	Server-Side	Additional Components
▪ User Interface (UI): text, images, menus ▪ Logic executed in the browser via HTML, CSS, JavaScript	▪ Web Server routes requests (Apache, Nginx, Microsoft IIS) ▪ Application server runs business logic ▪ Database Server: MySQL, PostgreSQL, MongoDB, Redis	▪ APIs & database management system ▪ Security: auth, encryption, security tokens, and cookies ▪ Caching, load balancers, monitoring

WEB DEVELOPMENT FUNDAMENTALS

Communication Fundamentals

Client-Server Model

หลักการพื้นฐานที่สุดของการสื่อสารบนเว็บ

Client-Server คือรูปแบบที่แบ่งบทบาทออกเป็น 2 ฝ่าย: Client ผู้ขอใช้บริการ และ Server ผู้ให้บริการ ทั้งสองฝั่งสื่อสารกันผ่านเครือข่าย โดยแยกกันพัฒนาและทำงานได้อย่างอิสระ



- 1 Client ริเริ่มการร้องขอ (Request) เสมอ
- 2 Server รอรับคำขอ ประมวลผล และตอบกลับ (Response) สามารถให้บริการหลาย Client พร้อมกัน
- 3 แยกอิสระ Client และ Server พัฒนา อัปเดต และ Deploy แยกจากกันได้ トラバไตที่ยังคุยกัันผ่าน Interface เดิม

HTTP Methods

คำสั่งมาตรฐานที่ Client ใช้บอก Server ว่าต้องการทำอะไรกับข้อมูล

GET

ขอข้อมูล

อ่านข้อมูลจาก Server โดยไม่
เปลี่ยนแปลงสิ่งใด

ตัวอย่าง

ดึงรายการสินค้า

POST

สร้างข้อมูลใหม่

ส่งข้อมูลไปสร้าง Resource ใหม่บน
Server

ตัวอย่าง

สร้างคำสั่งซื้อใหม่

PUT

แก้ไขข้อมูลทั้งหมด

แทนที่ข้อมูลเดิมทั้งก้อนด้วยข้อมูลใหม่

ตัวอย่าง

แก้ไขโปรไฟล์ผู้ใช้

DELETE

ลบข้อมูล

ลบ Resource ที่ระบุออกจาก Server

ตัวอย่าง

ลบสินค้าออกจากตะกร้า

HTTP Status Code

รหัส 3 หลักที่ Server ใช้บอกผลลัพธ์ของ Request กลับไปยัง Client

2xx

สำเร็จ

200 OK, 201 Created

3xx

เปลี่ยนเส้นทาง

301 Moved, 304 Not Modified

4xx

ผิดพลาดจาก Client

400 Bad Request,
401 Unauthorized,
404 Not Found

5xx

ผิดพลาดจาก Server

500 Internal Server Error,
503 Unavailable

ทำไม HTTP ถึงเป็น Stateless

Stateless หมายถึง Server จะไม่จดจำข้อมูลของ Request ก่อนหน้าเอาไว้เลย ทุก Request ที่ส่งมาต้องมีข้อมูลครบถ้วนในตัวเอง เสมือนเป็นคำขอใหม่ทุกครั้ง แม้จะมาจาก Client เดิมก็ตาม

ข้อดีของ Stateless

- Server ไม่ต้องเก็บข้อมูล Session ของทุก Client ทำให้ Scale ออกได้ง่าย (เพิ่มหลาย Instance ได้อิสระ)
- Request ใดล้มเหลว ก็ไม่กระทบ Request อื่น เพราะแต่ละคำขอเป็นอิสระต่อกัน

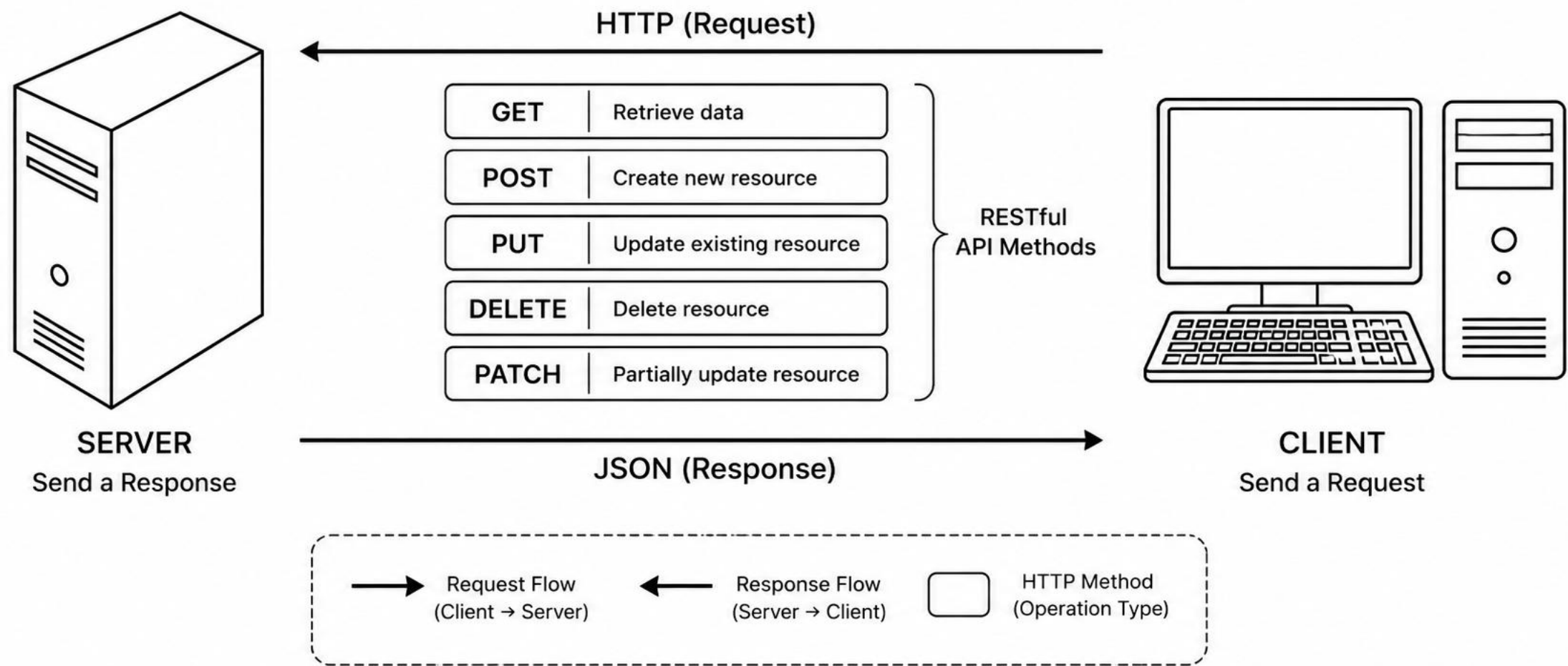
แล้ว Login ค้างได้อย่างไร?

แม้ Server ไม่จำ Client แต่ Client สามารถแนบข้อมูลยืนยันตัวตนไปกับทุก Request ได้เอง ผ่าน:

- Cookie เก็บ Session ID
- Token เช่น JWT แนบใน Header

REST (Representational State Transfer)

รูปแบบการเข้าถึงและจัดการข้อมูลระหว่าง Client และ Server





Document Object Modeling (DOM)

Asst.Prof. Dr. Umaporn Supasitthimethee

ผศ.ดร.อุมาพร สุภสีทธิเมธี



Learning Objective

- Understand the structure and purpose of the Document Object Model (DOM) in JavaScript.
- Identify and access HTML elements using JavaScript methods (e.g., getElementById, querySelector).
- Modify element properties, attributes, and content dynamically through JavaScript.
- Apply DOM manipulation to create interactive and responsive elements on a webpage.



Document Object Model (DOM)

- Understanding the Document Object Model in JavaScript
- Node Types
- Traversing Nodes
- Selecting Nodes
- Manipulating Nodes



What is a Document Object Modeling (DOM)?

- The Document Object Model (DOM) represents the HTML page so that programs can change the document structure, style, and content.
- The DOM represents a document as a **hierarchical tree of nodes**, allowing developers to select, add, remove, and modify individual parts of the page.
- The DOM is now a truly cross-platform, language-independent way of representing and manipulating pages for markup.
- The DOM is not a programming language, but without it, the JavaScript language wouldn't have any model or notion of web pages, HTML documents, XML documents, and their component parts.



<script> Tag Placement

```
<html>
  <head>
    <title>Example HTML Page</title>
    <script src="script.js"></script>

  </head>
  <body>
    <!-- content here -->
  </body>
</html>
```

- Including all JavaScript files in the `<head>` of a document means that all of the JavaScript code must be downloaded, parsed, and interpreted before the page begins rendering



<script> Tag Placement

```
<html>
  <head>
    <title>Example HTML Page</title>
  </head>
  <body>
    <!-- content here -->
    <script src="script.js"></script>
  </body>
</html>
```

- For pages that **require a lot of JavaScript code**, this can cause a noticeable delay in page rendering, during which time the browser will be completely blank.
- For this reason, modern web applications typically include all JavaScript references in the `<body>` element, after the page content.



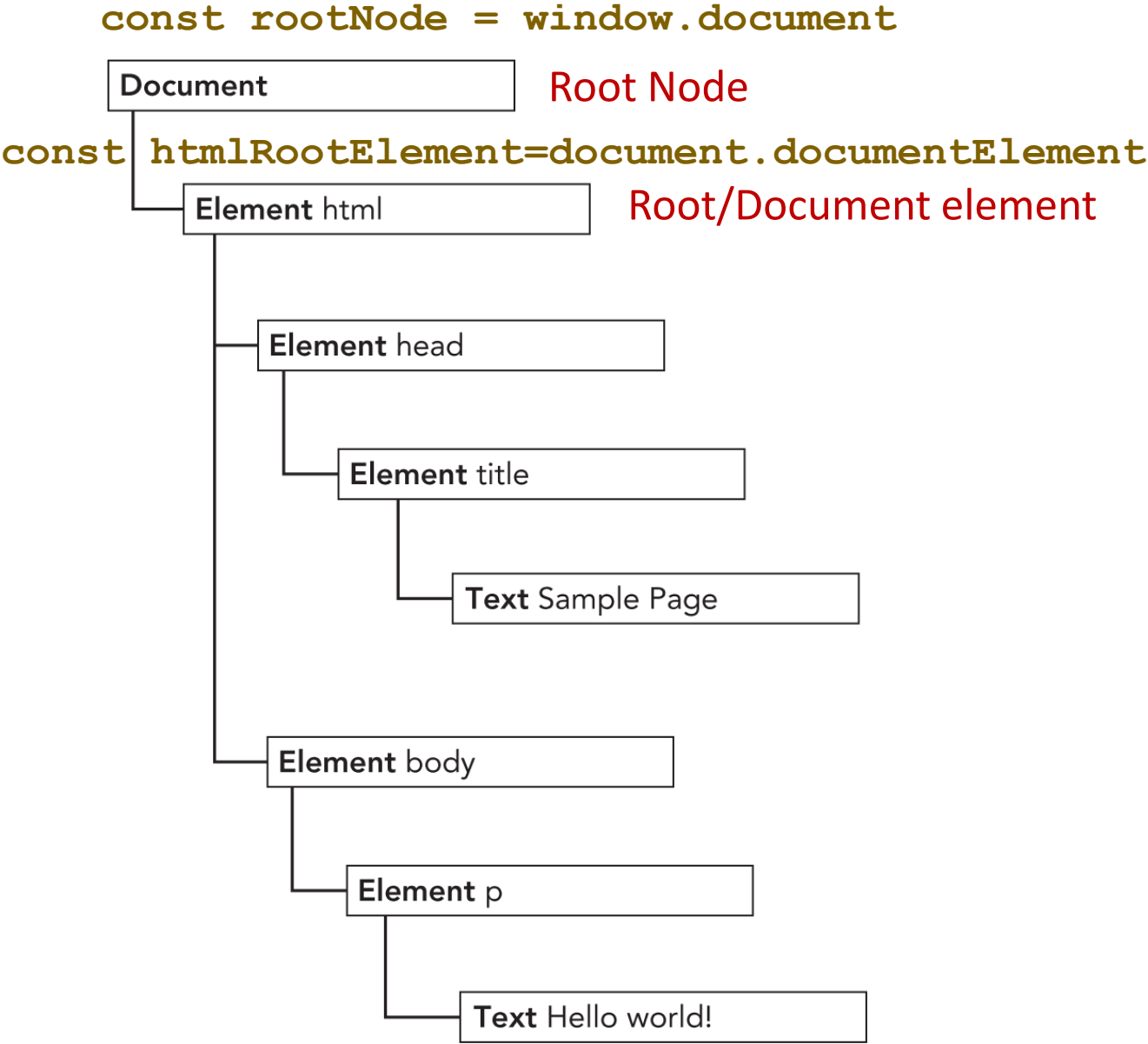
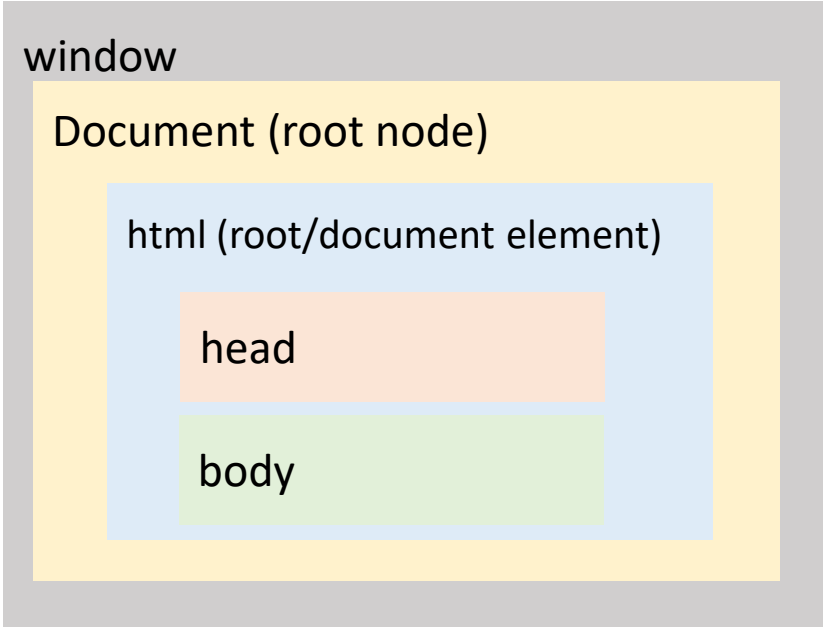
Document Object Modeling (DOM)

- When an HTML document is loaded into a web browser, it becomes a **document object**, everything is a **node**
- The document object is the root node of the HTML document and all other nodes: element nodes, text nodes, attribute nodes, and comment nodes
- Each node type has different characteristics, data, and methods, and each may have relationships with other nodes.
- These relationships create a hierarchy that allows markup to be represented as a tree, rooted at a particular node.



Dom and JavaScript

```
<html>
  <head>
    <title>Sample Page</title>
  </head>
  <body>
    <p>Hello World!</p>
  </body>
</html>
```



In browsers, the document object is a property of window and so is accessible globally.
It is read-only and represents the entire HTML page.

Document Children

- This example has no other elements come before `<html>`. It shows that the values of `documentElement`, `firstChild`, and `childNodes[0]` are all the same—all three point to the `<html>` element.

```
<html>
  <head>

  </head>
  <body>

  </body>
</html>
```

```
const html = document.documentElement // get reference to <html>
alert(html === document.childNodes[0]) // true
alert(html === document.firstChild) // true
```

```
Const head = document.head // get reference to <head>
const body = document.body // get reference to <body>
```



Nodes Types

Node types are represented by one of the following 12 numeric constants

`Node.nodeType` Read only

Returns an `unsigned short` representing the type of the node. Possible values are:

Name	Value
ELEMENT_NODE	1
ATTRIBUTE_NODE	2
TEXT_NODE	3
CDATA_SECTION_NODE	4
PROCESSING_INSTRUCTION_NODE	7
COMMENT_NODE	8
DOCUMENT_NODE	9
DOCUMENT_TYPE_NODE	10
DOCUMENT_FRAGMENT_NODE	11

Root node

- The Dom **Node** interface is implemented in JavaScript as the Node type, which is accessible in all browsers
- All node types inherit from Node in JavaScript, so **all node types share the same basic properties and methods.**
- Every node has **nodeName**, **nodeValue** **nodeType** **property** that indicates the type of node that it is.

<https://developer.mozilla.org/en-US/docs/Web/API/Node>

```
if (someNode.nodeType === Node.ELEMENT_NODE) {  
    alert("Node is an element.");  
}
```

Node.ENTITY_REFERENCE_NODE (5), Node.ENTITY_NODE (6), and Node.NOTATION_NODE (12) have been deprecated and are not in use anymore



Node	Value of nodeValue	nodeName value
CDATASection	Content of the CDATA section	"#cdata-section"
Comment	Content of the comment	"#comment"
Document	null	"#document"
DocumentFragment	null	"#document-fragment"
DocumentType	null	The value of DocumentType.name
Element	null	The value of Element.tagName
ProcessingInstruction	Entire content excluding the target	The value of ProcessingInstruction.target
Text	Content of the text node	"#text"

The nodeName, nodeType, nodeValue Properties

//index.html

```
<div id="exampleElement" style="font-size: 10pt">  
  This is some <span style="display: none">hidden</span> text content.  
</div>
```

//main.js

```
const divElement =  
document.getElementById('exampleElement')  
const divChild = divElement.childNodes  
divChild.forEach((child) => {  
  console.log(child.nodeName)  
  console.log(child.nodeType)  
  console.log(child.nodeValue)  
})
```

//output

```
#text  
3  
This is some  
SPAN  
1  
Null  
#text  
3  
Text content.
```

* Child nodes include elements, text and comments.



Element

- New elements can be created by using the `document.createElement()` method.
- This method accepts a single argument, which is the tag name of the element to create.
- In HTML documents, the tag name is case-insensitive.
- To create a `<div>` element, the following code can be used:

```
let div = document.createElement("div");
```

- The element can be added to the document tree using `appendChild()`, `insertBefore()`, `replaceChild()`, `removeChild()`.
 - The following code adds the newly created element to the document's `<body>` element:
- ```
document.body.appendChild(div);
```
- Once the element has been added to the document tree, the browser renders it immediately. Any changes to the element after this point are immediately reflected by the browser.

# Attr

- The Attr node inherits from Node but is not considered a part of the document tree.
- Common Node attributes like parentNode, previousSibling, and nextSibling are `null` for an Attr node. You can, however, get the element to which the attribute belongs with the `ownerElement` property.

```
<div id="exampleElement" style="font-size: 10pt">
 This is some hidden text content. //index.html
</div>
```

```
const element = document.getElementById('exampleElement') //main.js
console.log(element.nodeType) //1
console.log(element.attributes[0]) //id="exampleElement"
console.log(element.attributes[0].nodeType) //2
console.log(element.attributes[0].value) //exampleElement
console.log(element.attributes[0].ownerElement)
// <div id="exampleElement" style="font-size: 10pt">...</div>
const divStyleAttr = element.getAttribute('style')
console.log(divStyleAttr) //font-size: 10pt
if (element.hasAttribute("bar")) {
 // do something
}
```

# Listing All Attributes on an Element

```
// Target element: <input id="user" type="text" value="Alex" disabled>

const input = document.getElementById('user')
const allAttributes = input.getAttributeNames()

allAttributes.forEach((name) => {
 const value = input.getAttribute(name)
 console.log(`${name}: ${value}`)
})

// Outputs:
// id: user
// type: text
// value: Alex
// disabled:
```

# innerHTML, innerText, and textContent

innerHTML – คัดลอก **HTML ทั้งหมด**/parse และ render จริง/ ระวัง XSS (Cross-Site Scripting) ถ้ารับค่ามาจากผู้ใช้

```
<div id="demo">
 Hello
 World
</div>

<script>
 let el = document.getElementById("demo")
 console.log(el.innerHTML)
 // "Hello world" (ตัวหนาจริง)
</script>
```

innerText - คัดลอก**ข้อความที่มองเห็นจริง ๆ** บนหน้าเว็บ/ ละเว้นแท็ก HTML และยังประมวลผล CSS/ ใช้เมื่ออยากได้ ข้อความที่ผู้ใช้เห็นจริง ๆ

```
<div id="demo">
 Hello
 hidden
 world
</div>

<script>
 let el = document.getElementById("demo")
 console.log(el.innerText)
 // "Hello \n world" (ไม่มีhidden)
</script>
```

textContent – คัดลอก **ข้อความทั้งหมด** ที่อยู่ใน element/ จะรวมข้อความที่ซ่อนด้วย (display:none)/ ไม่สนใจ CSS/ ไม่ parse HTML

```
<div id="demo">
 Hello
 hidden
 world
</div>

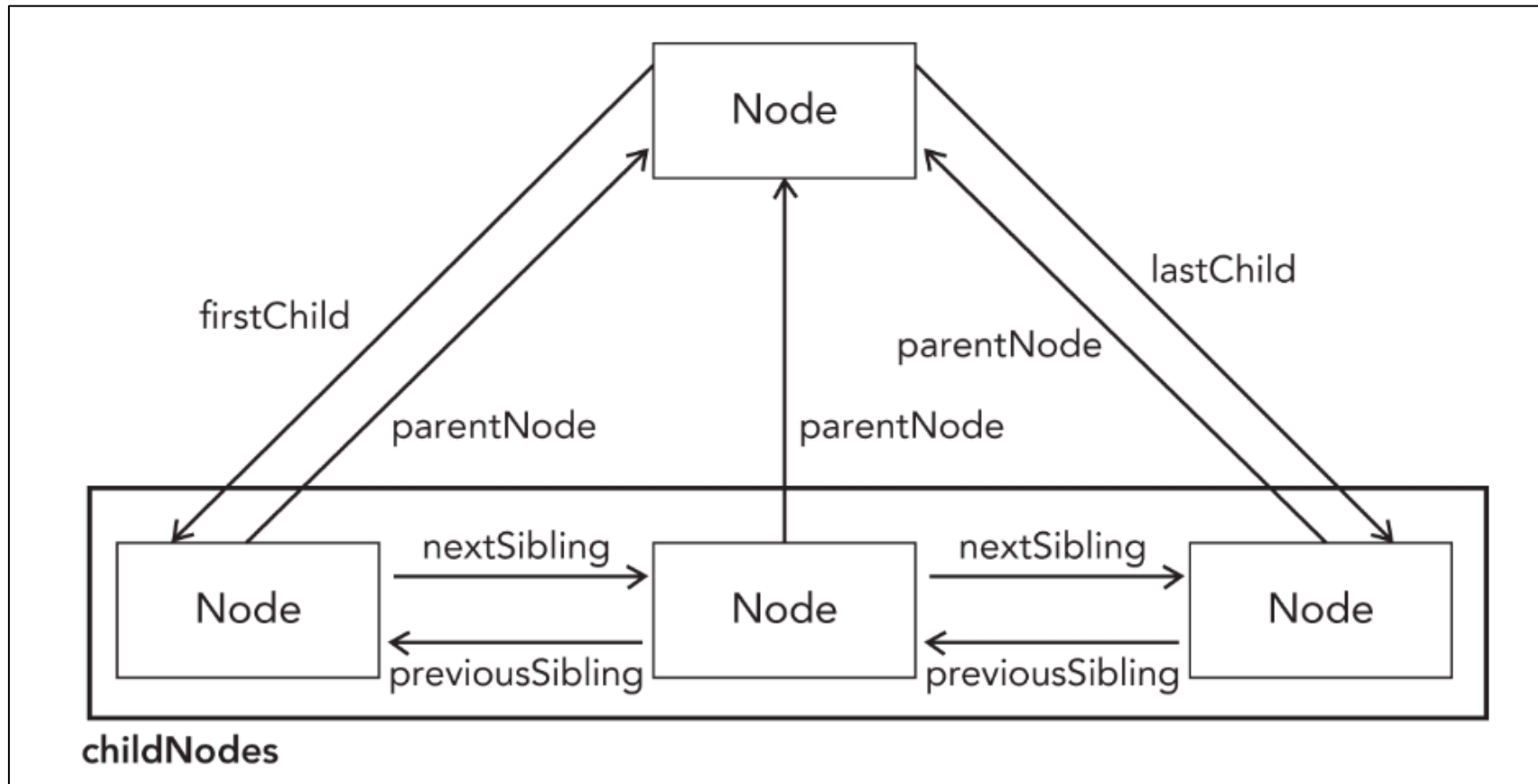
<script>
 let el = document.getElementById("demo")
 console.log(el.textContent)
 // "Hello hidden world"
</script>
```



# Traversing Nodes

## Node Relationships

The value of `someNode.firstChild` is always equal to `someNode.childNodes[0]`,  
and the value of `someNode.lastChild` is always equal to `someNode.childNodes[someNode.childNodes.length-1]`.



With all of these relationships, the **childNodes** property is really more of a convenience than a necessity because **it's possible to reach any node in a document tree** by simply using the relationship pointers.



# Element Traversal

**The Element Traversal API adds five new properties to DOM elements:**

- `childElementCount`—Returns the number of child elements (excludes text nodes and comments).
- `firstElementChild`—Points to the first child that is an element. Element-only version of `firstChild`.
- `lastElementChild`—Points to the last child that is an element. Element-only version of `lastChild`.
- `previousElementSibling`—Points to the previous sibling that is an element. Element-only version of `previousSibling`.
- `nextElementSibling`—Points to the next sibling that is an element. Element-only version of `nextSibling`.

# Element Traversal Example

```
<!DOCTYPE html>
<html lang="en">
 <head>
 <title>Manipulating Elements</title>
 </head>
 <body>
 <h3>SIT@KMUTT Restaurant</h3>
 <div class="menu">
 <ul id="appetizer">
 <li class="vegan">Vegetable Rolls
 <li class="meat">Chicken Wings
 <li class="meat">Tuna Sandwich

 <ul id="soup">
 <li class="meat">Spicy Bacon-Corn Soup
 <li class="vegan">Vegetable Soup
 <li class="meat">Beef Soup
 <li class="vegan">Coconut Soup

 </div>
 <p>***Enjoy Your Meal, Thank you***</p>
 </body>
</html>
```

```
const htmlElement = document.querySelector('html')
console.log(htmlElement.parentNode) // #document root node
console.log(htmlElement.parentElement) // null
```

```
const soupMenu = document.querySelector('#soup')

const firstChildNode = soupMenu.firstChild
const nextFirstChildNode =
 firstChildNode.nextElementSibling
const lastChildNode = soupMenu.lastElementChild
const previousLastChildNode =
 lastChildNode.previousElementSibling

console.log(firstChildNode)
// <li class="meat">Spicy Bacon-Corn Soup
console.log(lastChildNode)
// <li class="vegan">Coconut Soup
console.log(nextFirstChildNode)
// <li class="vegan">Vegetable Soup
console.log(previousLastChildNode)
// <li class="meat">Beef Soup

const meatMenus = document.querySelectorAll('.meat')
meatMenus.forEach((meatMenu) => console.log(meatMenu))
/* <li class="meat">Chicken Wings
 <li class="meat">Tuna Sandwich
 <li class="meat">Spicy Bacon-Corn Soup
 <li class="meat">Beef Soup */
```

# Element Traversal Example

```
<!DOCTYPE html>
<html lang="en">
 <head>
 <title>Manipulating Elements</title>
 </head>
 <body>
 <h3>SIT@KMUTT Restaurant</h3>
 <div class="menu">
 <ul id="appetizer">
 <li class="vegan">Vegetable Rolls
 <li class="meat">Chicken Wings
 <li class="meat">Tuna Sandwich

 <ul id="soup">
 <li class="meat">Spicy Bacon-Corn Soup
 <li class="vegan">Vegetable Soup
 <li class="meat">Beef Soup
 <li class="vegan">Coconut Soup

 </div>
 <p>***Enjoy Your Meal, Thank you***</p>
 </body>
</html>
```

```
let currentChildNode=soupMenu.firstElementChild
// <li class="meat">Spicy Bacon-Corn Soup

while (currentChildNode !== null) {
 console.log(currentChildNode)
 currentChildNode =
currentChildNode.nextElementSibling
}
```

```
<li class="meat">Spicy Bacon-Corn Soup
<li class="vegan">Vegetable Soup
<li class="meat">Beef Soup
<li class="vegan">Coconut Soup
```

\*The firstChild and lastChild return the first and last child of a node, which can be any node type including text node, comment node, and element node.

\*\*The firstElementChild and lastElementChild return the first and last child Element node.



# Selecting Nodes

# Selecting Elements

The `Document` type provides methods to specific element or sets of elements to perform certain operations.

```
getElementById() // returns the element if found, or null if an element with that ID doesn't exist.
```

**HTMLCollection** `getElementsByTagName()` and `getElementsByClassName()` are live.

When an underlying document is changed, the HTMLCollection will be updated automatically.

- `getElementsByTagName()`

//returns an HTMLCollection of elements with the given tag name.

- `getElementsByClassName()`

//returns A live HTMLCollection an array-like object of all child elements which have all the given class name(s).

## NodeList

- `getElementsByName()`

// returns a NodeList, which returns all elements that have a given name attribute.

- `querySelectorAll()`

//the CSS query and returns all matching nodes instead of just one. This method returns a static instance of NodeList.

- `querySelector()`

//a CSS query and returns the first descendant element that matches the pattern or null if there is no matching element.



# HTMLCollection Vs. NodeList

An **HTMLCollection** is always a **live collection of element nodes**.

- For example: If you add a `<li>` element to a list in the DOM, the list in the HTMLCollection will also change.
- A HTMLCollection items can be accessed by their name attribute, id attribute, or index number.

A **NodeList** is a **list of all nodes** including elements nodes, attribute nodes, and text nodes extracted from a document.

- A NodeList is most often a **static collection**. For example: If you add a `<li>` element to a list in the DOM, the list in the NodeList will not change.
  - NodeList items can only be accessed by their index numbers.
- Both an HTMLCollection object and a NodeList object are an array-like collections (lists) of nodes extracted from a document.

# HTMLCollection (live) Vs. NodeList (static)

```
<div id="sections">
 <div class="section">Section 1</div>
 <div class="section">Section 2</div>
 <div class="section">Section 3</div>
 <div class="section">Section 4</div>
 <!-- <div class="section">Section 5</div> -->
</div>
```

```
const sectionDiv = document.getElementById('sections')
const sectionsbySelectorAll = document.querySelectorAll('.section')
const sectionsByClassName = document.getElementsByClassName('section')
```

```
//create a new div section
const div = document.createElement('div')
div.className = 'section'
div.textContent = 'section 5'
sectionDiv.appendChild(div)
console.log(sectionsByClassName.length) //5
console.log(sectionsbySelectorAll.length) //4
```

# HTMLCollection and NodeList Examples

```
<!DOCTYPE html>
<html lang="en">
 <head>
 <title>Document</title>
 </head>
 <body>
 <div id="app">
 This is a Text Node
 <div>This is a div Element Node#1</div>
 <div>This is a div Element Node#2</div>
 </div>
 <script src="../main.js" />
 </body>
</html>
```

```
HTMLCollection(2) [div, div]
 <div>This is a div Element Node#1</div>
 <div>This is a div Element Node#2</div>
NodeList(5) [text, div, text, div, text]
 " This is a Text Node "
 <div>This is a div Element Node#1</div>
 ▶ #text
 <div>This is a div Element Node#2</div>
 ▶ #text
 >
```

```
//main.js
const myApp = document.getElementById('app')
const childrensHTML = myApp.children //return HTMLCollection
const childNodesList = myApp.childNodes //return NodeList
Array.from(childrensHTML).forEach((element) => console.log(element))
//HTMLCollection(2) [div, div], cannot use forEach directly and need to create array
childNodesList.forEach((element) => console.log(element))
//NodeList(5) [text, div, text, div, text] can use forEach
```

# Selecting Elements

```
<html lang="en">
 <head>
 <title>Document Object Model Sample Page</title>
 </head>
 <body>
 <h1 id="myHeading">Selecting Elements</h1>
 <h2 class="heading">Document Object Model</h2>
 <h2 class="heading">DOM</h2>
 <ul class="all-list-items">
 <li class="list-item">out of class
 <li class="list-item">[1] Single Element by ID:
 <li class="list-item">[2] Single Element by CSS selector:
 <li class="list-item">[3] Multiple Element by CSS selector:
 <li class="list-item">[4] Multiple Element by class (live):
 <li class="list-item">[5] Multiple Element by tag (live):

 <input type="radio" value="red" name="color" id="colorRed">
 <label for="colorRed">Red</label>

 <input type="radio" value="green" name="color" id="colorGreen">
 <label for="colorGreen">Green</label>

 <input type="radio" value="blue" name="color" id="colorBlue">
 <label for="colorBlue">Blue</label>

 <script src="main.js"></script>
 </body>
</html>
```

```
const domElementId = document.getElementById("myHeading")
const domByTagName = document.getElementsByTagName("ul")
const domByName = document.getElementsByName("color")
const domClassName = document.getElementsByClassName("list-item")
const domQuerySelector = document.querySelector(".heading")
const domQuerySelectorAll = document.querySelectorAll(".heading")
console.log(domElementId) //<h1 id="myHeading">Selecting Elements</h1>
console.log(domByTagName) //HTMLCollection(2) [ul.all-list-items,ul]
console.log(domByTagName.length) //2
console.log(domByTagName.item(0)) //<ul class="all-list-items">...
console.log(domByName) //NodeList(3) [input#colorRed, input#colorGreen, input#colorBlue]
console.log(domClassName) //HTMLCollection(6) [li.list-item, li.list-item, li.list-item, li.list-item, li.list-item, li.list-item]
console.log(domQuerySelector) //<h2 class="heading">Document Object Model</h2>
console.log(domQuerySelectorAll) //NodeList(2) [h2.heading, h2.heading]
```

► `h1#myHeading`

► `HTMLCollection(2) [ul.all-list-items, ul]`

`2`

► `ul.all-list-items`

► `NodeList(3) [input#colorRed, input#colorGreen, input#colorBlue]`

► `HTMLCollection(6) [li.list-item, li.list-item, li.list-item, li.list-item, li.list-item, li.list-item]`

► `h2.heading`

► `NodeList(2) [h2.heading, h2.heading]`

# Selector Examples (querySelector and querySelectorALL)

- To select all paragraph (p) elements in a document whose classes include warning or note, you can do the following:

```
const special = document.querySelectorAll("p.warning, p.note")
```

- You can also query by ID

```
const el = document.querySelector("#main, #basic, #exclamation")
```

After executing the above code, `el` contains the first element in the document whose ID is one of main, basic, or exclamation.

- You can select specific elements with ID or Class or Parent elements

```
const baseElement = document.querySelector("p")
```

```
//get the first element <p> in the document
```

```
const productList = document.querySelector("div[id='product-list']") // ("div#product-list")
```

```
// get <div> with the id "product-list"
```

```
const techProduct = document.querySelectorAll('div.tech')
```

```
// get a list of the document's <div> with the class "tech"
```

```
const techProduct = document.querySelectorAll('.tech')
```

```
// get a list of the document's element with the class "tech"
```

```
const techProductPElement = document.querySelectorAll('div.tech>p')
```

```
// get a list of the document's <p> elements whose immediate parent element is a <div> with the class "tech"
```

```
const techProductPElement = document.querySelectorAll('div.tech p')
```

```
// get a list of the document's <p> elements whose immediate ancestor element is a <div> with the class "tech"
```



# element.closest(selector)

Finds the **closest ancestor** of an element that matches the specified selector.

- Searches upward from the current element, moving through its parent elements until reaching the document.
- The closest ancestor Element or itself, which matches the selectors. If there is no such element, **null** is returned.

```
<article>
 <div id="div-01">
 Here is div-01
 <div id="div-02">
 Here is div-02
 <div id="div-03">Here is div-03</div>
 </div>
 </div>
</article>
```

```
const el = document.getElementById("div-03");

// the closest ancestor with the id of "div-02"
console.log(el.closest("#div-02")); // <div id="div-02">

// the closest ancestor which is a div in a div
console.log(el.closest("div div"));
// <div id="div-03">
// the closest ancestor which is a div and has a parent
// article
console.log(el.closest("article > div"));
// <div id="div-01">
```



# Manipulating Nodes



# Manipulating Nodes

- `appendChild()` // adds a newly node to the end of the `childNodes` list
- `createElement()` //create a new HTML element
- `insertBefore(newNode, referenceNode)` //The node to insert becomes the previous sibling of the reference node
- `replaceChild(newChild, oldChild)` //replaces a child node within the given (parent) node
- `removeChild(child)` //removes a child node from the DOM and returns the removed node.

# Using createElement, createAttribute, and createTextNode

```
const pElement = document.createElement('p')
const idAttr = document.createAttribute('id')
idAttr.value = 1001
pElement.setAttributeNode(idAttr)
// pElement.setAttribute('id', 1001)
const pText = document.createTextNode('Hello')
pElement.appendChild(pText)
// pElement.textContent = 'Hello'

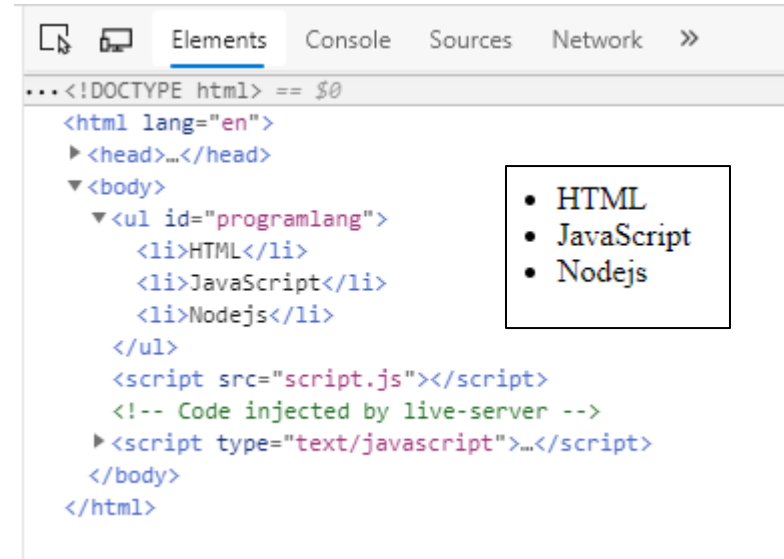
const divElement = document.querySelector('div')
divElement.appendChild(pElement)
```

```
console.log(pElement.parentElement) //div
console.log(idAttr.parentElement) //null
console.log(idAttr.ownerElement) //p#1001
console.log(pText.parentElement) //p#1101
```

```
▼ <div>
 <p id="1001">Hello</p>
</div>
```

# appendChild()

```
<html lang="en">
 <head>
 <title>Manipulating Elements</title>
 </head>
 <body>
 <ul id="programlang">
 </body>
</html>
```

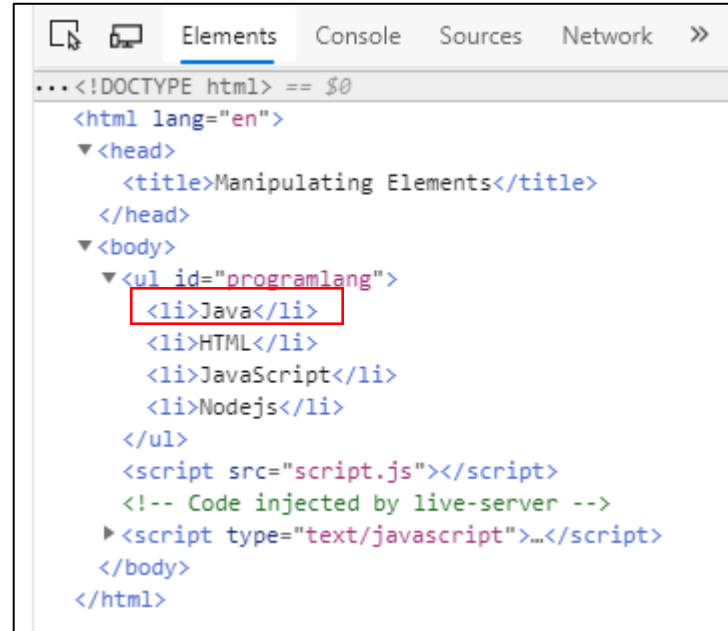


```
const langs = ["HTML", "JavaScript", "Nodejs"]
const langElement = document.querySelector("#programlang")
for (const lang of langs) {
 const li=document.createElement("li")
 li.textContent=lang
 langElement.appendChild(li)
}
```

# insertBefore()

```
const firstLangElement=langElement.firstElementChild
const newLangNode=document.createElement("li")
newLangNode.textContent="Java"
langElement.insertBefore(newLangNode, firstLangElement)
```

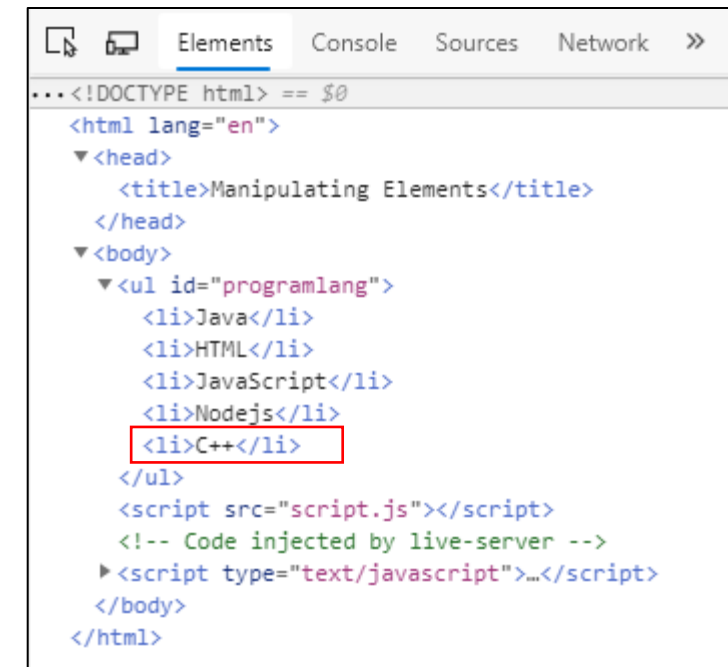
```
const newLastLangNode=document.createElement("li")
newLastLangNode.textContent="C++"
langElement.insertBefore(newLastLangNode, null)
```



```
...<!DOCTYPE html> == $0
<html lang="en">
 <head>
 <title>Manipulating Elements</title>
 </head>
 <body>
 <ul id="programlang">
 Java
 HTML
 JavaScript
 Nodejs

 <script src="script.js"></script>
 <!-- Code injected by live-server -->
 <script type="text/javascript">...</script>
 </body>
</html>
```

- Java
- HTML
- JavaScript
- Nodejs



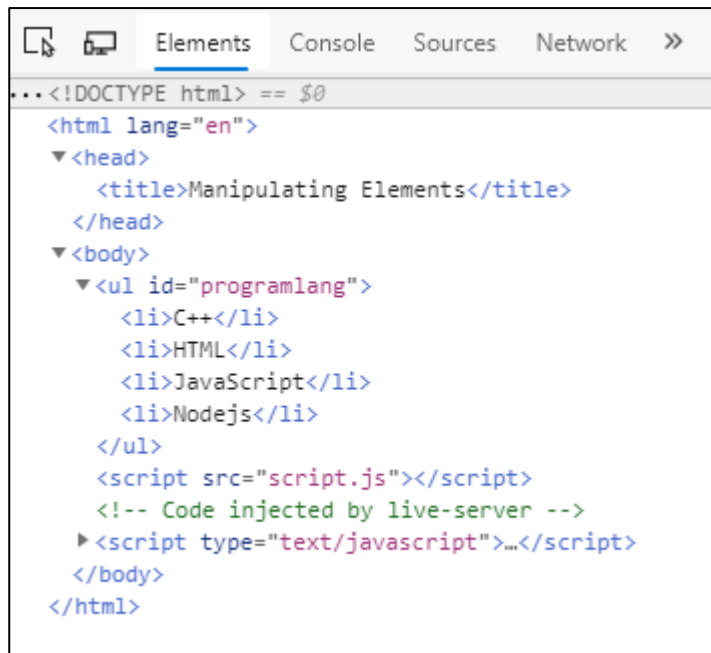
```
...<!DOCTYPE html> == $0
<html lang="en">
 <head>
 <title>Manipulating Elements</title>
 </head>
 <body>
 <ul id="programlang">
 Java
 HTML
 JavaScript
 Nodejs
 C++

 <script src="script.js"></script>
 <!-- Code injected by live-server -->
 <script type="text/javascript">...</script>
 </body>
</html>
```

- Java
- HTML
- JavaScript
- Nodejs
- C++

# replaceChild(), removeChild()

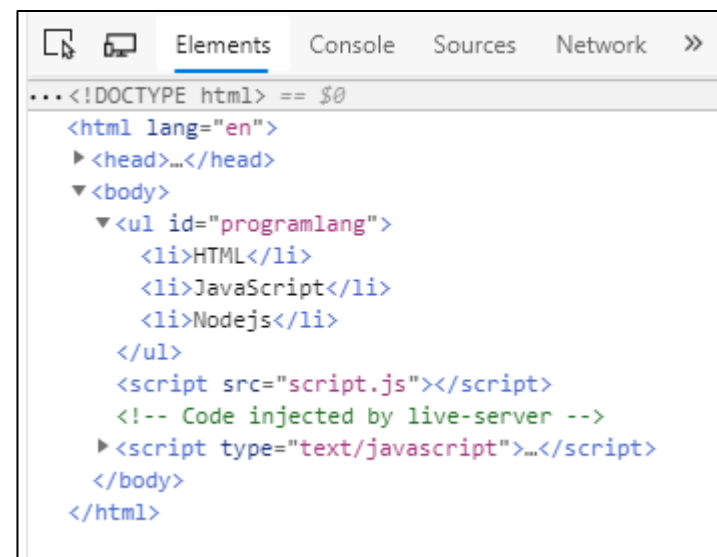
```
langElement.replaceChild(newLastLangNode, newLangNode)
```



The screenshot shows the 'Elements' panel of a browser's developer tools. The DOM tree is expanded to show a `<ul id="programlang">` element containing four `<li>` elements: `C++`, `HTML`, `JavaScript`, and `Nodejs`. The `C++` element is highlighted.

- C++
- HTML
- JavaScript
- Nodejs

```
langElement.removeChild(langElement.firstChild)
```



The screenshot shows the 'Elements' panel after the first child has been removed. The `<ul id="programlang">` element now contains only three `<li>` elements: `HTML`, `JavaScript`, and `Nodejs`. The `C++` element is no longer present.

- ~~C++~~
- HTML
- JavaScript
- Nodejs

# DocumentFragment: Reducing How Often You Touch the Real DOM

- DocumentFragment is a 'temporary holding area' in memory — it isn't part of the actual page
- Create all the elements into the fragment first, then appendChild the fragment into the real DOM 'once'
- The end result is exactly the same, but Reflow/Repaint happens only once instead of on every iteration of the loop

```
const fragment = document.createDocumentFragment()
const list = document.getElementById('list')

for (let i = 0; i < 1000; i++) {
 const li = document.createElement("li")
 li.textContent = "Item " + i;
 fragment.appendChild(li) // hasn't touched the real DOM yet
}
list.appendChild(fragment) // touches the real DOM just once!
```



# dataset and classList



# dataset

- In JavaScript, the dataset property is a DOM (Document Object Model) interface that provides a way to access and manipulate custom data attributes (attributes starting with data-) on HTML elements.
- It exposes a map of strings(DOMStringMap) with an entry for each data-\* attribute.

```
<div data-name="box" data-id="123" data-color="Red"></div>
```

- It offers a convenient and structured approach to store and retrieve data directly within your HTML, making it accessible to your JavaScript code.
- When accessing these attributes through the dataset property in JavaScript, the attribute names are automatically converted from kebab-case (e.g., data-my-attribute) to camelCase (e.g., element.dataset.myAttribute).



# Dataset Example

```
const button = document.getElementById('modal-button')
// Accessing data attributes
const buttonDataset = button.dataset //DOMStringMap {modalId: 'modal1', userStatus: 'active'}

for (const key in buttonDataset) {
 console.log(`name: ${key}, value: ${buttonDataset[key]}`)
 //name: modalId, value: modal1
 //name: userStatus, value: inactive
 //name: color, value: green
}
console.log(button.dataset.modalId) // modal1
//console.log(button.getAttribute('data-modal-id')) // modal1
console.log(button.dataset.userStatus) // active

// add a data attribute
button.dataset.color = 'green'
//button.setAttribute('data-color', 'green')

// Modifying a data attribute
button.dataset.color = 'orange'
console.log(button.dataset.color) // orange

button.dataset.userStatus = 'inactive'
console.log(button.dataset['userStatus']) // inactive

// Removing a data attribute
button.removeAttribute('data-modal-id')
console.log(button.dataset.modalId) // undefined
```

```
<button id="modal-button" data-modal-id="modal1" data-user-status="inactive">Open Modal</button>
```



# classList

- The classList property in the DOM (Document Object Model) is a read-only property of an HTML element that returns a DOMTokenList object (Iterable Object).
- This DOMTokenList represents the collection of class names assigned to that element's class attribute.
- The classList property provides a convenient way to manipulate an element's CSS classes dynamically using JavaScript, without directly modifying the className string.



# Key features and methods

- **add(className1, className2, ...)** Adds one or more class names to the element's class list. If a class already exists, it will not be added again.
- **remove(className1, className2, ...)** Removes one or more class names from the element's class list.
- **toggle(className, force)** Toggles the presence of a class name. If the class is present, it's removed; if absent, it's added. The optional force argument (a boolean) can be used to explicitly add (if true) or remove (if false) the class, regardless of its current state.
- **contains(className)** Returns a boolean indicating whether the element's class list contains the specified class name.
- **replace(oldClassName, newClassName)** Replaces an existing class name with a new one.
- **length** A property that returns the number of class names in the list.



# Example

```
const divMenu = document.getElementById('menu')
const pElement = document.createElement('p')
// Add a class
pElement.classList.add('text-xl', 'font-semibold')
pElement.classList.add('p-5')
console.log(pElement.classList.length) //3
// Remove a class
pElement.classList.remove('p-5', 'text-xl')
// Toggle a class
pElement.classList.toggle('p-5')
// Replace a class
pElement.classList.replace('p-5', 'p-2')

// Check if a class exists
console.log(pElement.classList.length) //3
console.log(pElement.classList.contains('p-2')) //true
pElement.classList.forEach((className) => console.log(className)) //text-xl //font-semibold //p-2

pElement.textContent = 'Pumpkin soup'
divMenu.appendChild(pElement)
```

```
<p class="text-xl font-semibold p-2">Pumpkin soup</p>
```



# System Dialogs

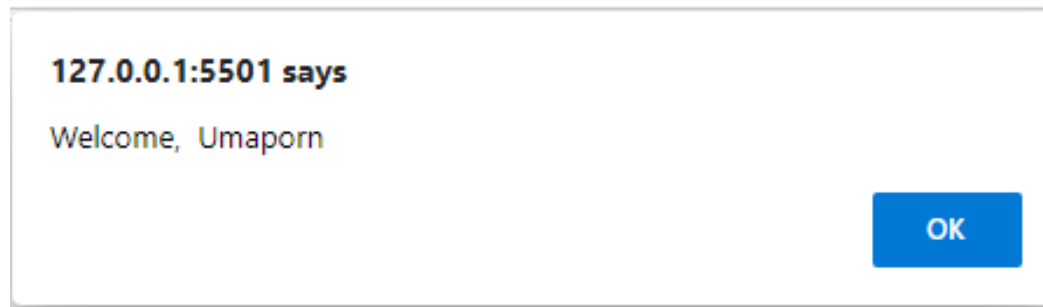


# System Dialogs

- The browser is capable of invoking system dialogs to display to the user through the **alert()**, **confirm()**, and **prompt()** methods.
- These dialogs are not related to the web page being displayed in the browser and do not contain HTML.
- Their appearance is determined by operating system and/or browser settings rather than CSS.
- Additionally, each of these dialogs is synchronous and modal, meaning code execution stops when a dialog is displayed, and resumes after it has been dismissed.

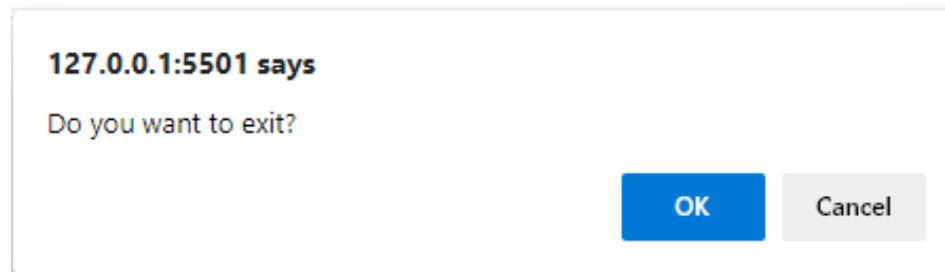
# System Dialogs: alert()

- The `alert()` method simply accepts a string to display to the user.
- When `alert()` is called, a system message box displays the specified text to the user, followed by a single OK button.
- Alert dialogs are typically used when users must be made aware of something that they have no control over, such as an error.



# System Dialogs: confirm()

- A `confirm()` method looks similar to an alert dialog in that it displays a message to the user.
- The main difference between the two is the **presence of a Cancel button along with the OK button**, which allows the user to indicate if a given action should be taken.
- The `confirm()` method returns `true` if the user clicked "OK", and `false` otherwise.



# System Dialogs: prompt()

- The final type of dialog is displayed by calling `prompt()`, which along **with OK and Cancel buttons**, this dialog has a text box where the user may enter some data.
- The `prompt()` method accepts two arguments: the *text to display* to the user, and the *default value* for the text box (which can be an empty string).
- If the OK button is clicked, `prompt()` returns the value in the text box; if Cancel is clicked or the dialog is otherwise closed without clicking OK, the function returns `null`.

